```
// ============================================================================
//  Gestion des reservations de vols - Implementation C++ (POO)
//  Compilation : g++ -std=c++17 -Wall -Wextra -o reservation_vols reservation_vols.cpp
//  Execution   : ./reservation_vols        (Windows : reservation_vols.exe)
// ============================================================================

#include <algorithm>
#include <cctype>
#include <ctime>
#include <iomanip>
#include <iostream>
#include <memory>
#include <stdexcept>
#include <string>
#include <vector>

using namespace std;

// ============================================================================
//  Utilitaires : saisie securisee et validation
// ============================================================================
namespace Utils {

// Lancee quand le flux d'entree est ferme (Ctrl+D / Ctrl+Z)
struct EntreeFermee {};

string trim(const string& s) {
    size_t debut = 0, fin = s.size();
    while (debut < fin && isspace(static_cast<unsigned char>(s[debut]))) ++debut;
    while (fin > debut && isspace(static_cast<unsigned char>(s[fin - 1]))) --fin;
    return s.substr(debut, fin - debut);
}

string enMinuscules(string s) {
    transform(s.begin(), s.end(), s.begin(),
              [](unsigned char c) { return static_cast<char>(tolower(c)); });
    return s;
}

string dateDuJour() {
    time_t t = time(nullptr);
    tm* lt = localtime(&t);
    char buf[16];
    strftime(buf, sizeof(buf), "%d/%m/%Y", lt);
    return string(buf);
}

// Format attendu : JJ/MM/AAAA
bool dateValide(const string& d) {
    if (d.size() != 10 || d[2] != '/' || d[5] != '/') return false;
    for (int i : {0, 1, 3, 4, 6, 7, 8, 9})
        if (!isdigit(static_cast<unsigned char>(d[i]))) return false;
    int j = stoi(d.substr(0, 2));
    int m = stoi(d.substr(3, 2));
    int a = stoi(d.substr(6, 4));
    if (a < 2000 || a > 2100 || m < 1 || m > 12) return false;
    static const int joursParMois[] = {31, 29, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
    return j >= 1 && j <= joursParMois[m - 1];
}

string lireLigne(const string& invite) {
    cout << invite;
    string ligne;
    if (!getline(cin, ligne)) throw EntreeFermee();
    return trim(ligne);
}

// Texte obligatoire (non vide)
string lireTexte(const string& invite) {
    while (true) {
        string s = lireLigne(invite);
        if (!s.empty()) return s;
```

```
        cout << "  [Erreur] Ce champ ne peut pas etre vide.\n";
    }
}

int lireEntier(const string& invite, int min, int max) {
    while (true) {
        string s = lireLigne(invite);
        try {
            size_t pos = 0;
            int v = stoi(s, &pos);
            if (pos != s.size()) throw invalid_argument("reste");
            if (v < min || v > max) {
                cout << "  [Erreur] Entrez un nombre entre " << min << " et " << max << ".\n";
                continue;
            }
            return v;
        } catch (const exception&) {
            cout << "  [Erreur] Saisie invalide : entrez un nombre entier.\n";
        }
    }
}

float lireReel(const string& invite, float min) {
    while (true) {
        string s = lireLigne(invite);
        try {
            size_t pos = 0;
            float v = stof(s, &pos);
            if (pos != s.size()) throw invalid_argument("reste");
            if (v < min) {
                cout << "  [Erreur] La valeur doit etre >= " << min << ".\n";
                continue;
            }
            return v;
        } catch (const exception&) {
            cout << "  [Erreur] Saisie invalide : entrez un nombre (ex: 85000).\n";
        }
    }
}

string lireDate(const string& invite) {
    while (true) {
        string s = lireLigne(invite);
        if (dateValide(s)) return s;
        cout << "  [Erreur] Date invalide. Format attendu : JJ/MM/AAAA.\n";
    }
}

}  // namespace Utils

// ============================================================================
//  Classe Passager
// ============================================================================
class Passager {
private:
    int idPassager;
    string nom;
    string prenom;
    string numeroPasseport;  // unique
    string telephone;

public:
    Passager(int id, const string& nom, const string& prenom,
             const string& numeroPasseport, const string& telephone)
        : idPassager(id), nom(nom), prenom(prenom),
          numeroPasseport(numeroPasseport), telephone(telephone) {
        if (nom.empty() || prenom.empty() || numeroPasseport.empty())
            throw invalid_argument("Nom, prenom et numero de passeport sont obligatoires.");
    }

    int getId() const { return idPassager; }
    const string& getNom() const { return nom; }
```

```
    const string& getPrenom() const { return prenom; }
    const string& getNumeroPasseport() const { return numeroPasseport; }
    const string& getTelephone() const { return telephone; }

    void enregistrer() const {
        cout << "  -> Passager enregistre avec l'ID " << idPassager << " ("
             << prenom << " " << nom << ").\n";
    }

    void afficher() const {
        cout << "  Passager #" << idPassager << " : " << prenom << " " << nom
             << " | Passeport : " << numeroPasseport
             << " | Tel : " << telephone << "\n";
    }
};

// ============================================================================
//  Classe Vol
// ============================================================================
class Vol {
private:
    int idVol;
    string villeDepart;
    string villeArrivee;
    string dateDepart;
    float prix;
    int capacite;
    int placesDisponibles;  // invariant : 0 <= placesDisponibles <= capacite

public:
    Vol(int id, const string& depart, const string& arrivee, const string& date,
        float prix, int capacite)
        : idVol(id), villeDepart(depart), villeArrivee(arrivee), dateDepart(date),
          prix(prix), capacite(capacite), placesDisponibles(capacite) {
        if (depart.empty() || arrivee.empty())
            throw invalid_argument("Les villes de depart et d'arrivee sont obligatoires.");
        if (Utils::enMinuscules(depart) == Utils::enMinuscules(arrivee))
            throw invalid_argument("La ville de depart et d'arrivee doivent etre differentes.");
        if (!Utils::dateValide(date))
            throw invalid_argument("Date de depart invalide (format JJ/MM/AAAA).");
        if (prix < 0) throw invalid_argument("Le prix ne peut pas etre negatif.");
        if (capacite <= 0) throw invalid_argument("La capacite doit etre superieure a 0.");
    }

    int getId() const { return idVol; }
    const string& getVilleDepart() const { return villeDepart; }
    const string& getVilleArrivee() const { return villeArrivee; }
    const string& getDateDepart() const { return dateDepart; }
    float getPrix() const { return prix; }
    int getCapacite() const { return capacite; }

    int verifierPlacesDisponibles() const { return placesDisponibles; }
    bool estComplet() const { return placesDisponibles == 0; }

    // Operation interdite si le vol est complet
    void reserverPlace() {
        if (estComplet()) throw runtime_error("Operation interdite : le vol est complet.");
        --placesDisponibles;
    }

    void libererPlace() {
        if (placesDisponibles < capacite) ++placesDisponibles;
    }

    void creerVol() const {
        cout << "  -> Vol cree avec l'ID " << idVol << " : " << villeDepart << " -> "
             << villeArrivee << " le " << dateDepart << ".\n";
    }

    void afficher() const {
        cout << "  Vol #" << idVol << " : " << villeDepart << " -> " << villeArrivee
             << " | Depart : " << dateDepart
```

```
            << " | Prix : " << fixed << setprecision(0) << prix << " FCFA"
            << " | Places : " << placesDisponibles << "/" << capacite
            << (estComplet() ? "  [COMPLET]" : "") << "\n";
    }
};

// ============================================================================
//  Classe Paiement
// ============================================================================
class Paiement {
private:
    int idPaiement;
    float montant;
    string datePaiement;
    string modePaiement;
    string statutRemboursement;  // "Aucun" ou "Rembourse"

public:
    Paiement(int id, float montant, const string& mode)
        : idPaiement(id), montant(montant), datePaiement(Utils::dateDuJour()),
          modePaiement(mode), statutRemboursement("Aucun") {
        if (montant < 0) throw invalid_argument("Le montant ne peut pas etre negatif.");
        if (mode.empty()) throw invalid_argument("Le mode de paiement est obligatoire.");
    }

    int getId() const { return idPaiement; }
    float getMontant() const { return montant; }
    bool estRembourse() const { return statutRemboursement == "Rembourse"; }

    void enregistrerPaiement() const {
        cout << "  -> Paiement #" << idPaiement << " de " << fixed << setprecision(0)
             << montant << " FCFA enregistre (" << modePaiement << ").\n";
    }

    void traiterRemboursement() {
        if (estRembourse()) throw runtime_error("Operation interdite : paiement deja rembourse.");
        statutRemboursement = "Rembourse";
        cout << "  -> Remboursement de " << fixed << setprecision(0) << montant
             << " FCFA traite pour le paiement #" << idPaiement << ".\n";
    }

    void afficher() const {
        cout << "    Paiement #" << idPaiement << " | " << fixed << setprecision(0) << montant
             << " FCFA | " << modePaiement << " | " << datePaiement
             << " | Remboursement : " << statutRemboursement << "\n";
    }
};

// ============================================================================
//  Classe Reservation  (associe un Passager, un Vol et 0..1 Paiement)
// ============================================================================
class Reservation {
private:
    int idReservation;
    string dateReservation;
    string statut;  // "En attente", "Confirmee", "Annulee"
    Passager* passager;             // association (non proprietaire)
    Vol* vol;                       // association (non proprietaire)
    unique_ptr<Paiement> paiement;  // composition 0..1

public:
    Reservation(int id, Passager* p, Vol* v)
        : idReservation(id), dateReservation(Utils::dateDuJour()),
          statut("En attente"), passager(p), vol(v) {
        if (!p || !v) throw invalid_argument("Passager et vol obligatoires.");
    }

    int getId() const { return idReservation; }
    const string& getStatut() const { return statut; }
    const Passager& getPassager() const { return *passager; }
    const Vol& getVol() const { return *vol; }
    bool estActive() const { return statut != "Annulee"; }
```

```
    bool aPaiement() const { return paiement != nullptr; }

    // Vrai si le meme passager a deja une reservation active sur le meme vol
    bool verifierDoublon(const vector<unique_ptr<Reservation>>& existantes) const {
        for (const auto& r : existantes)
            if (r.get() != this && r->estActive() &&
                r->passager == passager && r->vol == vol)
                return true;
        return false;
    }

    void creerReservation() {
        vol->reserverPlace();  // peut lever une exception si complet
        cout << "  -> Reservation #" << idReservation << " creee (statut : " << statut << ").\n";
    }

    // Enregistre le paiement et confirme la reservation
    void enregistrerPaiement(int idPaiement, const string& mode) {
        if (statut == "Annulee")
            throw runtime_error("Operation interdite : reservation annulee.");
        if (paiement)
            throw runtime_error("Operation interdite : cette reservation est deja payee.");
        paiement = make_unique<Paiement>(idPaiement, vol->getPrix(), mode);
        statut = "Confirmee";
        paiement->enregistrerPaiement();
    }

    void annulerReservation() {
        if (statut == "Annulee")
            throw runtime_error("Operation interdite : reservation deja annulee.");
        if (paiement && !paiement->estRembourse()) paiement->traiterRemboursement();
        vol->libererPlace();
        statut = "Annulee";
        cout << "  -> Reservation #" << idReservation << " annulee, place liberee.\n";
    }

    // Annuler le paiement = rembourser puis annuler la reservation
    void annulerPaiement() {
        if (!paiement) throw runtime_error("Operation interdite : aucun paiement pour cette
        reservation.");
        if (paiement->estRembourse())
            throw runtime_error("Operation interdite : paiement deja annule/rembourse.");
        annulerReservation();
    }

    void afficher() const {
        cout << "  Reservation #" << idReservation << " | Date : " << dateReservation
             << " | Statut : " << statut << "\n";
        cout << "  "; passager->afficher();
        cout << "  "; vol->afficher();
        if (paiement) paiement->afficher();
        else cout << "    Aucun paiement enregistre.\n";
    }
};

// ============================================================================
//  Classe AgenceVols : gere les collections (vector) et les operations metier
// ============================================================================
class AgenceVols {
private:
    vector<unique_ptr<Passager>> passagers;
    vector<unique_ptr<Vol>> vols;
    vector<unique_ptr<Reservation>> reservations;
    int prochainIdPassager = 1, prochainIdVol = 1;
    int prochainIdReservation = 1, prochainIdPaiement = 1;

    Passager* trouverPassager(int id) const {
        for (const auto& p : passagers) if (p->getId() == id) return p.get();
        throw runtime_error("Passager introuvable (ID " + to_string(id) + ").");
    }
    Vol* trouverVol(int id) const {
        for (const auto& v : vols) if (v->getId() == id) return v.get();
```

```
        throw runtime_error("Vol introuvable (ID " + to_string(id) + ").");
    }
    Reservation* trouverReservation(int id) const {
        for (const auto& r : reservations) if (r->getId() == id) return r.get();
        throw runtime_error("Reservation introuvable (ID " + to_string(id) + ").");
    }

public:
    // 1. Enregistrer un passager
    void enregistrerPassager() {
        string nom = Utils::lireTexte("  Nom : ");
        string prenom = Utils::lireTexte("  Prenom : ");
        string passeport = Utils::lireTexte("  Numero de passeport : ");
        string tel = Utils::lireTexte("  Telephone : ");

        for (const auto& p : passagers)
            if (Utils::enMinuscules(p->getNumeroPasseport()) == Utils::enMinuscules(passeport))
                throw runtime_error("Operation interdite : ce numero de passeport existe deja.");

        passagers.push_back(make_unique<Passager>(prochainIdPassager++, nom, prenom, passeport, tel));
        passagers.back()->enregistrer();
    }

    // 2. Ajouter un vol
    void ajouterVol() {
        string depart = Utils::lireTexte("  Ville de depart : ");
        string arrivee = Utils::lireTexte("  Ville d'arrivee : ");
        string date = Utils::lireDate("  Date de depart (JJ/MM/AAAA) : ");
        float prix = Utils::lireReel("  Prix (FCFA) : ", 0);
        int capacite = Utils::lireEntier("  Capacite (nombre de places) : ", 1, 1000);

        vols.push_back(make_unique<Vol>(prochainIdVol++, depart, arrivee, date, prix, capacite));
        vols.back()->creerVol();
    }

    // 3. Afficher les vols disponibles
    void afficherVolsDisponibles() const {
        bool trouve = false;
        for (const auto& v : vols)
            if (!v->estComplet()) { v->afficher(); trouve = true; }
        if (!trouve) cout << "  Aucun vol disponible pour le moment.\n";
    }

    // 4. Rechercher un vol (par villes, recherche partielle, champ vide = tous)
    void rechercherVol() const {
        string depart = Utils::enMinuscules(Utils::lireLigne("  Ville de depart (vide = toutes) : "));
        string arrivee = Utils::enMinuscules(Utils::lireLigne("  Ville d'arrivee (vide = toutes) : "));
        bool trouve = false;
        for (const auto& v : vols) {
            bool okDepart = Utils::enMinuscules(v->getVilleDepart()).find(depart) != string::npos;
            bool okArrivee = Utils::enMinuscules(v->getVilleArrivee()).find(arrivee) != string::npos;
            if (okDepart && okArrivee) { v->afficher(); trouve = true; }
        }
        if (!trouve) cout << "  Aucun vol ne correspond a la recherche.\n";
    }

    // 5. Effectuer une reservation
    void effectuerReservation() {
        if (passagers.empty()) throw runtime_error("Aucun passager enregistre. Utilisez d'abord l'option
        1.");
        if (vols.empty()) throw runtime_error("Aucun vol disponible. Utilisez d'abord l'option 2.");

        int idP = Utils::lireEntier("  ID du passager : ", 1, 1000000);
        Passager* p = trouverPassager(idP);
        int idV = Utils::lireEntier("  ID du vol : ", 1, 1000000);
        Vol* v = trouverVol(idV);

        auto r = make_unique<Reservation>(prochainIdReservation, p, v);
        if (r->verifierDoublon(reservations))
            throw runtime_error("Operation interdite : ce passager a deja reserve ce vol.");
        r->creerReservation();  // leve une exception si le vol est complet
        ++prochainIdReservation;
```

```
        reservations.push_back(move(r));
    }

    // 6. Enregistrer un paiement
    void enregistrerPaiement() {
        int idR = Utils::lireEntier("  ID de la reservation : ", 1, 1000000);
        Reservation* r = trouverReservation(idR);
        cout << "  Montant a payer : " << fixed << setprecision(0) << r->getVol().getPrix() << " FCFA\n";
        string mode = Utils::lireTexte("  Mode de paiement (Espece, Mobile Money, Carte...) : ");
        r->enregistrerPaiement(prochainIdPaiement, mode);
        ++prochainIdPaiement;
    }

    // 7. Annuler un paiement
    void annulerPaiement() {
        int idR = Utils::lireEntier("  ID de la reservation : ", 1, 1000000);
        trouverReservation(idR)->annulerPaiement();
    }

    // 8. Consulter les reservations
    void consulterReservations() const {
        if (reservations.empty()) { cout << "  Aucune reservation enregistree.\n"; return; }
        for (const auto& r : reservations) {
            cout << "  #" << setw(3) << r->getId() << " | "
                 << r->getPassager().getPrenom() << " " << r->getPassager().getNom() << " | "
                 << r->getVol().getVilleDepart() << " -> " << r->getVol().getVilleArrivee()
                 << " | " << r->getStatut() << "\n";
        }
    }

    // 9. Afficher les informations d'une reservation
    void afficherReservation() const {
        int idR = Utils::lireEntier("  ID de la reservation : ", 1, 1000000);
        trouverReservation(idR)->afficher();
    }
};

// ================================================================================
//  Menu principal
// ================================================================================
void afficherMenu() {
    cout << "\n=========== GESTION DES RESERVATIONS DE VOLS ===========\n"
         << "  1. Enregistrer un passager\n"
         << "  2. Ajouter un vol\n"
         << "  3. Afficher les vols disponibles\n"
         << "  4. Rechercher un vol\n"
         << "  5. Effectuer une reservation\n"
         << "  6. Enregistrer un paiement\n"
         << "  7. Annuler un paiement\n"
         << "  8. Consulter les reservations\n"
         << "  9. Afficher les informations d'une reservation\n"
         << " 10. Quitter\n"
         << "=========================================================\n";
}

int main() {
    AgenceVols agence;
    try {
        while (true) {
            afficherMenu();
            int choix = Utils::lireEntier("Votre choix : ", 1, 10);
            if (choix == 10) break;
            try {
                switch (choix) {
                    case 1: agence.enregistrerPassager(); break;
                    case 2: agence.ajouterVol(); break;
                    case 3: agence.afficherVolsDisponibles(); break;
                    case 4: agence.rechercherVol(); break;
                    case 5: agence.effectuerReservation(); break;
                    case 6: agence.enregistrerPaiement(); break;
                    case 7: agence.annulerPaiement(); break;
                    case 8: agence.consulterReservations(); break;
```

```
                    case 9: agence.afficherReservation(); break;
                }
            } catch (const Utils::EntreeFermee&) {
                throw;  // on remonte la fermeture du flux
            } catch (const exception& e) {
                cout << "  [Erreur] " << e.what() << "\n";
            }
        }
    } catch (const Utils::EntreeFermee&) {
        cout << "\n";
    }
    cout << "Au revoir !\n";
    return 0;
}
```